THE PORTFOLIO ENGINE

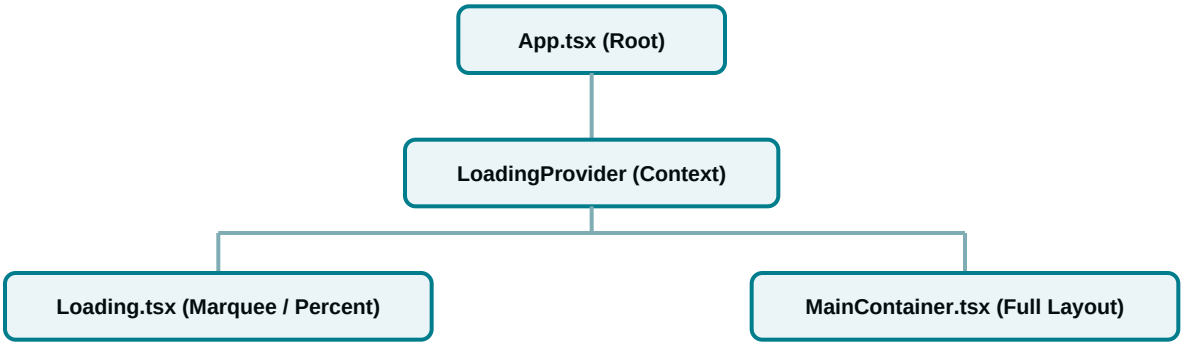
An Absolute Beginner's Blueprint & Reference Manual

A Complete Guide to Code Architecture, Layout Reactivity, and Web Animation

Target Audience:	Web Designers, Developers, and Beginners
Subject Project:	Manohar Kendhuli - Interactive Portfolio Website
Framework System:	React 18 / TypeScript / Vite
Primary Animation Tool:	GSAP (GreenSock Animation Platform)
Document Version:	2.0.0 (Production Release)

1. High-Level Blueprint & Structure

To understand this project, imagine you are building a physical gallery house. The layout must guide visitors step-by-step through a loading hallway, into the main lobby, and past several exhibit boards. In software, this is achieved by orchestrating several files together. The diagram below shows the basic structural nesting layout of your portfolio:



A. The Entry Pipeline Sequence

When a user opens the webpage, the following steps occur:
1. **Mounting the App**: The browser loads index.html, boots the runtime scripts in main.tsx, and renders the **App.tsx** component.
2. **Locking Scroll & Preloading**: The **LoadingProvider.tsx** context initializes and renders **Loading.tsx**. This component acts as a lock. It sets the body overflow styling to overflow: hidden, completely blocking the user from scrolling. In the background, a loading percentage count-up ticks from 0% to 100%.
3. **Entrance Transition**: As soon as the count reaches 100%, the loader calls a function inside **initialFX.ts**. This function performs the opening animation. It unlocks body scrolling, splits the heading texts into letters using GSAP's SplitText, and slides them up with motion blur to reveal the main screen.

B. The Main Page Single-Page Layout

Once the loader finishes, the screen renders **MainContainer.tsx**. The sections of the website are arranged vertically. Instead of jumping abruptly, the desktop browser initializes **GSAP ScrollSmoother** inside **Navbar.tsx**. This creates momentum-based inertia scrolling—making the screen scroll like silk. On mobile, this is bypassed for standard mobile touch scrolls.

2. Page-by-Page & File-by-File Technical Guide

This section breaks down every primary code file in the project, detailing exactly what it is responsible for and how it coordinates with the rest of the application.

File / Path Name	Role in Portfolio	How it Works
App.tsx	The entry controller root.	Wraps the app in the LoadingProvider context, and lazy-loads the MainContainer con
MainContainer.tsx	Page layout manager.	Tracks screen width. Decides whether to mount responsive desktop elements or mob
Navbar.tsx	Header menu & ScrollSmoother.	Renders menu links. Initializes GSAP ScrollSmoother for momentum physics on desk
Loading.tsx	Preloader page element.	Animates a horizontal text marquee and counts from 0% to 100% using a random inc
Cursor.tsx	Custom cursor follower.	Creates a trailing circle div. Tweaks cursor shape/opacity using data-cursor elements
SocialIcons.tsx	Magnetic links panel.	Aligns your social links. Snaps the links toward the mouse using coordinate distance
About.tsx	About me profile details.	Renders your profile image and highlights your professional background and achieve
WhatIDo.tsx	Interactive skills categories.	Splits frontend & backend skills. Binds click tap-handlers on mobile and hover states
Career.tsx	Historical experience path.	Displays your career events chronologically. Coordinates with timeline border animati
Work.tsx	Horizontal portfolio showcase.	Pins the page using ScrollTrigger and moves project boxes horizontally on vertical sc
TechStack.tsx	Visual technology cards.	Displays your specific dev tools and languages in styled grid containers with glow effe
Contact.tsx	Footer contact card.	Polished bottom block displaying your email, phone, and copyright details.

3. Interactive Physics & Animation Mathematics

To make the site look premium, we write physics-like behaviors in JavaScript. This section explains the formulas and code logic behind the interactive elements.

A. Linear Interpolation (Lerping) for Custom Cursor

A regular cursor moves instantly. The custom circle follower trails the cursor smoothly. This is done using **linear interpolation (lerp)**. Each frame, we move the custom cursor only a fraction of the distance to the real mouse coordinates:

```
// Lerping Math Formula currentX += (mouseX - currentX) / delay; currentY += (mouseY - currentY) / delay;
```

In **Cursor.tsx**, this loop runs inside requestAnimationFrame. By setting delay = 6, the circle takes 6 frames to ease to the mouse position, creating a smooth visual delay.

B. Distance-Based Magnetic Pull

Inside **SocialIcons.tsx**, we calculate the distance between the mouse pointer and the icon. If the mouse enters a close perimeter, the button snaps toward the mouse. When the mouse moves away, the button snaps back using the same easing logic:

```
// Checking hover boundaries dynamically const onMouseMove = (e: MouseEvent) => { const x = e.clientX - rect.left; const y = e.clientY - rect.top; if (x < 40 && x > 10 && y < 40 && y > 5) { mouseX = x; mouseY = y; } else { mouseX = rect.width / 2; // return to center mouseY = rect.height / 2; } };
```

C. Horizontal ScrollTrigger Viewport Pinning

Inside **Work.tsx**, when a desktop user scrolls, the page stops moving vertically. Instead, the project row translates horizontally. This is done by linking GSAP ScrollTrigger to CSS properties:

```
// Setup horizontal scroll trigger let timeline = gsap.timeline({ scrollTrigger: { trigger: ".work-section", start: "top top", end: `+=${translateX}`, scrub: true, pin: true, } }); timeline.to(".work-flex", { x: -translateX, ease: "none" });
```

D. Mobile Layout Adaptations & Tap Hooks

Because mobile touch devices do not support mouse cursors or hovers, the code disables the ScrollTrigger pinning and translates layouts to column grids. On mobile screens (width <= 1024px), **WhatIDo.tsx** automatically disables hover animations and binds a touch click listener to let users expand cards by tap.

4. Local Server Hosting & Deployment Operations

This section details how to host the project on your machine for development and how to push it live.

A. Local Development Server

To start editing the code and test changes locally, run the command:
npm run dev
This hosts the website on **http://localhost:5173/**. It uses Hot Module Replacement (HMR) to inject updates immediately into the open browser tab when a file is saved.

B. Production Build Compiling

To compile the source code into static HTML, CSS, and JS files ready for hosting, run:
npm run build
Vite compresses the assets, compiles the TypeScript files, and packages everything into the **dist/** directory.

C. Static Server Preview

To test the built production files locally (which is best for mobile network testing as it doesn't depend on real-time WebSockets), run:
npx vite preview --host
This serves the static build on port **4173** and binds to your local network IP (e.g. http://192.168.29.82:4173/) allowing phones on the same network to connect.

D. Live Production Deployment (Vercel)

We use Vercel's CLI to push the static folder directly onto Vercel servers for public hosting. The command is:
npx vercel --prod
Vercel uploads the compiled files, runs its cloud build pipeline, and aliases the deployment to your live domain (**https://portfolio-website-manoharmks-projects.vercel.app**) making it permanently live online.